CHAPITRE 4 — PASSAGE DU MCD AU MLD (CIF)

Par Robert DIASSÉ

A. INTRODUCTION

Le modèle conceptuel de données (MCD) permet de représenter la réalité métier de manière indépendante de toute technologie. Cependant, pour pouvoir implémenter une base de données dans un SGBD relationnel, il est nécessaire de traduire ce modèle conceptuel vers une structure logique.

Cette étape intermédiaire constitue le **Modèle Logique de Données (MLD)**.

Évolution du modèle de données :

```
┌──────────────────────────────────────────────────────────────┐
│                     ÉVOLUTION DU MODÈLE                      │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  MCD (Conceptuel)  →  MLD (Logique)  →  MPD (Physique)       │
│                                                              │
│  • Entités         →  • Relations    →  • Tables             │
│  • Associations    →  • Clés étrang. →  • Index              │
│  • Cardinalités    →  • Contraintes  →  • Triggers           │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Le passage du MCD au MLD consiste à :

- transformer les **entités** en **relations** (tables)
- traduire les **associations** selon leurs cardinalités
- définir les **clés primaires** et **clés étrangères**
- formaliser les **contraintes d'intégrité fonctionnelle**
- obtenir un **schéma relationnel** cohérent et prêt pour l'implémentation

B. DISTINCTION ENTRE MCD ET MLD

I. Le MCD (Modèle Conceptuel de Données)

Le MCD est :

- **conceptuel** : indépendant de toute technologie
- **orienté métier** : utilise le vocabulaire des utilisateurs
- **abstrait** : représente ce que le système doit modéliser

Il s'exprime avec des **entités**, des **associations** et des **cardinalités**.

Question centrale : *Que représente le système d'information ?*

II. Le MLD (Modèle Logique de Données)

Le MLD est :

- **logique** : basé sur le modèle relationnel
- **structuré** : organisé en relations (tables)
- **technique** : préparé pour l'implémentation

Il s'exprime avec des **relations**, des **clés primaires** et des **clés étrangères**.

Question centrale : *Comment structurer ces données dans une base relationnelle ?*

III. Correspondance terminologique

MCD	MLD	Notation
Entité	Relation (Table)	NOM_TABLE
Attribut	Attribut (Colonne)	nom_attribut
Identifiant	Clé primaire	# ou souligné
Association	Clé étrangère ou Table associative	→ ou FK
Occurrence	Ligne (Tuple)	-

C. RÈGLES DE TRANSFORMATION MCD → MLD

Le passage du MCD au MLD repose sur trois règles systématiques basées sur l'analyse des cardinalités :

```
╔═══════════════════════════════════════════════════════════╗
║           RÈGLES DE TRANSFORMATION MCD → MLD              ║
╚═══════════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════════╗
║                                                           ║
║  Règle 1 : Toute entité devient une relation              ║
║            Identifiant → Clé primaire                     ║
║                                                           ║
║  Règle 2 : Association selon cardinalités :               ║
║            • (1,n)-(1,1) → Clé étrangère                  ║
║            • (n,n)       → Table associative              ║
║            • (1,1)-(1,1) → Fusion ou clé étrangère        ║
║                                                           ║
║  Règle 3 : Dépendances fonctionnelles → CIF               ║
║                                                           ║
╚═══════════════════════════════════════════════════════════╝
```

D. TRANSFORMATION DES ENTITÉS

Règle fondamentale

Toute entité du MCD devient une relation dans le MLD.

L'identifiant de l'entité devient la clé primaire de la relation.

Exemple de transformation :

```
MCD :
┌──────────────────┐
│     CLIENT       │
├──────────────────┤
│ id_client        │  ← Identifiant
│ nom              │
│ adresse          │
└──────────────────┘

        ↓  Transformation

MLD :
┌──────────────────────────────────┐
│ CLIENT                           │
├──────────────────────────────────┤
│ id_client    (Clé primaire)      │
│ nom                              │
│ adresse                          │
└──────────────────────────────────┘

Notation textuelle  ou Schéma Relationnel:
Client(#id_client, nom, adresse)
```

Convention : La clé primaire est notée avec le symbole # ou soulignée.

E. TRANSFORMATION DES ASSOCIATIONS

La transformation dépend des **cardinalités maximales** de l'association.

I. Association de type (1,n) — (1,1)

Principe : La clé primaire de l'entité du côté (1,n) migre vers l'entité du côté (1,1) comme clé étrangère.

> (Le fort aide le faible en lui donnant son identifiant comme clès étrangère)

Règle mnémotechnique : *La clé migre toujours vers le côté où il y a le max a "1".*

Exemple de transformation :

```
MCD :
┌──────────┐          Passer        ┌──────────────┐
│  CLIENT  │ 1,n ──────────── 1,1   │  COMMANDE    │
├──────────┤                        ├──────────────┤
│id_client │                        │num_commande  │
│nom       │                        │date_commande │
│adresse   │                        └──────────────┘
└──────────┘

        ↓  Transformation

MLD :
┌────────────────────┐          ┌──────────────────────────┐
│ CLIENT             │          │ COMMANDE                 │
├────────────────────┤          ├──────────────────────────┤
│ id_client (CP)     │◄─────────│ num_commande (CP)        │
│ nom                │          │ date_commande            │
│ adresse            │          │ id_client (CE)           │
└────────────────────┘          └──────────────────────────┘

Notation textuelle  ou Schéma Relationnel:
Client(#id_client, nom, adresse)
Commande(#num_commande, date_commande, id_client)
```

II. Association de type (n,n)

Principe : L'association devient une nouvelle relation (table associative).

Règles de construction :

- La clé primaire est **composée** des clés primaires des deux entités
- Les attributs de l'association deviennent attributs de cette nouvelle relation

Exemple de transformation :

```
MCD :
┌──────────┐            Suivre        ┌──────────┐
│ ÉTUDIANTS │ 1,n ──────────── 1,n     │  COURS   │
├──────────┤          (note)          ├──────────┤
│id_etudiant│                          │id_cours  │
│nom       │                          │intitule  │
│prenom    │                          │nb_heures │
└──────────┘                          └──────────┘

        ↓  Transformation

MLD :
┌──────────────┐      ┌──────────────────────────┐      ┌──────────┐
│ ÉTUDIANT     │      │ INSCRIPTION              │      │ COURS    │
├──────────────┤      ├──────────────────────────┤      ├──────────┤
```

```
│ id_etudiant  │◄────│ id_etudiant (CP + CE)       │────►│ id_cours    │
│ (CP)         │     │ id_cours (CP + CE)          │     │ (CP)        │
│ nom          │     │ note                        │     │ intitule    │
│ prenom       │     └─────────────────────────────┘     │ nb_heures   │
└──────────────┘                                         └─────────────┘


Notation textuelle  ou Schéma Relationnel:
Étudiant(#id_etudiant, nom, prenom)
Cours(#id_cours, intitule, nb_heures)
Inscription(#id_etudiant, #id_cours, note)
```

III. Association de type (1,1) — (1,1)

Deux possibilités selon le contexte métier :

Option 1 - Fusion des entités (si elles représentent le même objet métier) :

```
Personne(#id_personne, nom, prenom, num_passeport, date_emission)
```

Option 2 - Ajout d'une clé étrangère (généralement du côté optionnel (0,1)) :

```
Personne(#id_personne, nom, prenom)
Passeport(#num_passeport, date_emission, id_personne)
```

IV. Cas des cardinalités minimales

Les cardinalités minimales (0 ou 1) n'affectent pas la structure du MLD, mais influencent les contraintes d'implémentation :

- **(0,n)** : participation optionnelle → clé étrangère nullable
- **(1,n)** : participation obligatoire → clé étrangère NOT NULL

F. CONTRAINTES D'INTÉGRITÉ FONCTIONNELLE (CIF)

I. Définition et rôle

Les **Contraintes d'Intégrité Fonctionnelle** garantissent que les dépendances fonctionnelles identifiées lors de la normalisation sont respectées dans le modèle logique.

Elles assurent la **cohérence**, la **validité** et le **respect des règles métier**.

```
╔═══════════════════════════════════════════════════════╗
║        LES QUATRE TYPES DE CONTRAINTES D'INTÉGRITÉ    ║
╚═══════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════╗
║                                                       ║
║  1. INTÉGRITÉ D'ENTITÉ                                ║
```

```
    ╟     → Clé primaire unique et non nulle                    ╟
    ╟                                                           ╟
    ╟  2. INTÉGRITÉ RÉFÉRENTIELLE                               ╟
    ╟     → Clé étrangère référence une clé primaire            ╟
    ╟                                                           ╟
    ╟  3. INTÉGRITÉ FONCTIONNELLE                               ╟
    ╟     → Respect des dépendances fonctionnelles              ╟
    ╟                                                           ╟
    ╟  4. INTÉGRITÉ DE DOMAINE                                  ╟
    ╟     → Valeurs conformes au domaine défini                 ╟
    ╟                                                           ╟
    ╙───────────────────────────────────────────────────────────╜
```

II. Types de contraintes d'intégrité

1. Intégrité d'entité

Définition : Chaque relation doit avoir une clé primaire unique et non nulle.

Règles :

- Aucune valeur de clé primaire en double (unicité)
- Aucune valeur de clé primaire nulle (non nullité)

Exemple :

```
VALIDE :
┌──────────────────────┐
│ CLIENT               │
├──────────────────────┤
│ id_client = 1001     │  ← Unique et non NULL
│ nom = "Dupont"       │
│ adresse = "Paris"    │
└──────────────────────┘

INVALIDE :
┌──────────────────────┐
│ CLIENT               │
├──────────────────────┤
│ id_client = NULL     │  ← NULL interdit pour CP
│ nom = "Martin"       │
│ adresse = "Lyon"     │
└──────────────────────┘
```

2. Intégrité référentielle

Définition : Toute clé étrangère doit référencer une clé primaire existante dans la relation référencée.

Règle : Une valeur de clé étrangère doit soit :

- correspondre à une clé primaire existante
- être nulle (si la participation est optionnelle)

Exemple d'intégrité respectée :

```
CLIENT                    COMMANDE
┌──────────────┐          ┌──────────────────────┐
│ id_client=1  │◄─────────│ num_commande = 101   │
│ nom=Dupont   │          │ date = 2026-01-15    │
│ adresse=...  │          │ id_client = 1        │ ✓ Existe dans CLIENT
└──────────────┘          └──────────────────────┘
```

Actions en cas de violation :

- **RESTRICT** : interdire l'opération
- **CASCADE** : propager la suppression/modification
- **SET NULL** : mettre à NULL les clés étrangères concernées
- **SET DEFAULT** : affecter une valeur par défaut

3. Intégrité fonctionnelle

Définition : Les dépendances fonctionnelles identifiées doivent être respectées.

Exemple : Si num_article -> designation, alors pour un même `num_article`, la désignation doit être identique.

Cette contrainte est naturellement respectée si l'article est correctement modélisé comme entité avec `num_article` comme clé primaire.

4. Intégrité de domaine

Définition : Chaque attribut ne peut prendre que des valeurs appartenant à son domaine de définition.

Exemples de domaines :

```
┌──────────────────────────────────────────────────┐
│  EXEMPLES DE DOMAINES D'ATTRIBUTS                │
├──────────────────────────────────────────────────┤
│                                                  │
│  age           : INTEGER BETWEEN 0 AND 150       │
│  email         : VARCHAR(100) FORMAT email       │
│  date_naissance : DATE < CURRENT_DATE            │
│  prix          : DECIMAL(10,2) >= 0              │
│  quantite      : INTEGER > 0                     │
│  sexe          : CHAR(1) IN ('M', 'F')           │
│  code_postal   : CHAR(5) PATTERN '[0-9]{5}'      │
│                                                  │
└──────────────────────────────────────────────────┘
```

G. EXEMPLE COMPLET DE TRANSFORMATION

I. MCD de départ

Contexte métier : Système de gestion de commandes clients.

```
┌──────────┐            Passer          ┌──────────────┐
│  CLIENT  │ 1,n ──────────── 1,1      │  COMMANDE    │
├──────────┤                           ├──────────────┤
│id_client │                           │num_commande  │
│nom       │                           │date_commande │
│adresse   │                           └──────────────┘
└──────────┘                                  │
                                              │ 1,n
                                              │
                                        Contenir
                                        (quantite)
                                              │
                                              │ 1,n
                                              ▼
                                        ┌──────────────┐
                                        │   ARTICLE    │
                                        ├──────────────┤
                                        │ref_article   │
                                        │designation   │
                                        │prix_unitaire │
                                        └──────────────┘
```

II. Application des règles de transformation

Étape 1 - Transformation des entités :

```
Client(#id_client, nom, adresse)
Commande(#num_commande, date_commande)
Article(#ref_article, designation, prix_unitaire)
```

Étape 2 - Association Client-Commande (1,n)-(1,1) :

Ajout de `id_client` dans Commande :

```
Commande(#num_commande, date_commande, id_client)
```

Étape 3 - Association Commande-Article (n,n) :

Création d'une table associative :

```
LigneCommande(#num_commande, #ref_article, quantite)
```

III. MLD final

Schéma relationnel complet :

```
┌──────────────────────┐
│ CLIENT               │
├──────────────────────┤
│ id_client (CP)       │
│ nom                  │
│ adresse              │
└──────────────────────┘
         ▲
         │ référencé par
         │
┌────────┴─────────────────────┐
│ COMMANDE                     │
├──────────────────────────────┤
│ num_commande (CP)            │
│ date_commande                │
│ id_client (CE)               │
└──────────────────────────────┘
         │
         │ référencé par
         │
         ▼
┌──────────────────────────────┐          référence          ┌────────────────────┐
│ LIGNE_COMMANDE               │──────────────────────────►  │ ARTICLE            │
├──────────────────────────────┤                             ├────────────────────┤
│ num_commande (CP + CE)       │                             │ ref_article (CP)   │
│ ref_article (CP + CE)        │◄────────────────────────────│ designation        │
│ quantite                     │                             │ prix_unitaire      │
└──────────────────────────────┘                             └────────────────────┘
```

Notation textuelle ou Schéma Relationnel:

```
Client(#id_client, nom, adresse)

Commande(#num_commande, date_commande, id_client)
  avec : id_client → Client.id_client

Article(#ref_article, designation, prix_unitaire)

LigneCommande(#num_commande, #ref_article, quantite)
  avec : num_commande → Commande.num_commande
         ref_article → Article.ref_article
```

IV. Contraintes d'intégrité associées

CIF1 - Intégrité d'entité :

- `id_client` est unique et non nul dans CLIENT
- `num_commande` est unique et non nul dans COMMANDE
- `ref_article` est unique et non nul dans ARTICLE
- `(num_commande, ref_article)` est unique dans LIGNE_COMMANDE

CIF2 - Intégrité référentielle :

- `id_client` dans COMMANDE référence CLIENT.id_client
- `num_commande` dans LIGNE_COMMANDE référence COMMANDE.num_commande
- `ref_article` dans LIGNE_COMMANDE référence ARTICLE.ref_article

CIF3 - Intégrité fonctionnelle :

- id_client -> nom, adresse
- num_commande -> date_commande, id_client
- ref_article -> designation, prix_unitaire
- (num_commande, ref_article) -> quantite

CIF4 - Intégrité de domaine :

- `quantite` > 0 (entier positif)
- `prix_unitaire` ≥ 0 (décimal positif)
- `date_commande` ≤ date du jour
- `nom`, `adresse`, `designation` : chaînes non vides

H. VÉRIFICATION DU MLD

Avant de valider le modèle logique, il est impératif de vérifier plusieurs aspects :

```
┌─────────────────────────────────────────────────────────────┐
│  CHECKLIST DE VALIDATION DU MLD                             │
├─────────────────────────────────────────────────────────────┤
│                                                             │
│  STRUCTURE :                                                │
│    □ Chaque relation possède une clé primaire               │
│    □ Toutes les CE référencent des CP existantes            │
│    □ Les cardinalités du MCD sont respectées                │
│    □ Aucune relation n'est redondante                       │
│                                                             │
│  NORMALISATION :                                            │
│    □ Première forme normale (1FN) respectée                 │
│    □ Aucune dépendance partielle (2FN)                      │
│    □ Aucune dépendance transitive (3FN)                     │
│    □ Pas de redondance introduite                           │
│                                                             │
│  CONTRAINTES :                                              │
```

```
│    □ Toutes les CIF sont identifiées                    │
│    □ Les règles métier sont traduites                   │
│    □ Les domaines sont définis                          │
│    □ Les actions référentielles sont spécifiées         │
│                                                         │
│  COHÉRENCE :                                            │
│    □ Toutes les DF du MCD sont préservées               │
│    □ Aucune information n'est perdue                    │
│    □ Le modèle est compréhensible                       │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

I. CONVENTIONS D'ÉCRITURE DU MLD

I. Notation standard du schéma relationnel

Format général :

```
┌─────────────────────────────────────────────────────────┐
│  FORMAT GÉNÉRAL DU SCHÉMA RELATIONNEL                   │
├─────────────────────────────────────────────────────────┤
│                                                         │
│  Relation(#clé_primaire, attribut1, attribut2,          │
│           clé_étrangère)                                │
│                                                         │
│  Légende :                                              │
│  • # ou souligné = clé primaire                         │
│  • → = référence vers                                   │
│  • CP = Clé Primaire                                    │
│  • CE = Clé Étrangère                                   │
│                                                         │
└─────────────────────────────────────────────────────────┘
```

II. Méthodes d'indication des clés étrangères

Méthode 1 - Notation simple :

```
Client(#id_client, nom, adresse)
Commande(#num_commande, date_commande, id_client)
```

Méthode 2 - Avec référence explicite :

```
Commande(#num_commande, date_commande,
         id_client → Client.id_client)
```

Méthode 3 - Avec notation formelle :

```
Client(#id_client, nom, adresse)
Commande(#num_commande, date_commande, id_client)
  avec : COMMANDE.id_client référence CLIENT.id_client
```

J. CONCLUSION

Le passage du MCD au MLD est une transformation **systématique et rigoureuse** qui suit des règles précises basées sur l'analyse des cardinalités et des dépendances fonctionnelles.

Points clés à retenir :

```
╔═══════════════════════════════════════════════════════╗
║              POINTS CLÉS À RETENIR                    ║
╚═══════════════════════════════════════════════════════╝
╔═══════════════════════════════════════════════════════╗
║                                                       ║
║  • Toute entité devient une relation                  ║
║  • Les cardinalités déterminent la transformation     ║
║  • Les CIF garantissent la cohérence des données      ║
║  • Le MLD préserve toutes les DF du MCD               ║
║  • Le modèle obtenu est prêt pour l'implémentation    ║
║                                                       ║
╚═══════════════════════════════════════════════════════╝
```

Un MLD correctement construit :

- respecte les **trois règles de transformation**
- préserve toutes les **dépendances fonctionnelles** identifiées dans le MCD
- formalise explicitement les **contraintes d'intégrité**
- garantit l'**absence de redondance**
- constitue une base solide pour l'**implémentation SQL**

Le MLD constitue ainsi le pont essentiel entre la modélisation conceptuelle orientée métier et l'implémentation physique dans un système de gestion de base de données relationnelle.